```
app.UseExceptionHandler("/Errors/MyCustomErrorPage");
```

The default setup has a built-in flaw. It is automatically assigned a layout page via the _ViewStart file that sits in the same folder. Consequently, if the exception is generated by an error condition within the layout, you will get a potentially recursive situation in which the exception occurs and the framework tries to display the error page, resulting in the exception again. For this reason, one of the first steps I usually take when starting a new application is setting the `Layout` to `null` in the error page and ensuring its content is not reliant on anything that could result in another exception being raised.

Listing 12.4 Setting the `Layout` property to `null` in the custom error page

```
@page
@model ErrorModel
@{
    Layout = null;                ❶
    ViewData["Title"] = "Error";
}
```

❶ Set Layout to null.

As might be obvious from its name, the exception-handler middleware only works when an unhandled exception—a server error—occurs within the application, resulting in an HTTP 500 status code. Other kinds of errors can occur while a user is visiting your application. For example, before you commented it out, you called the `NotFound` method if the city name passed into the City page could not be found in the database. You might recall from chapter 3 when we looked at handler method return types that the `NotFound` method is a helper result that returns a `NotFoundResult`. The `NotFoundResult` returns an HTTP 404 status code, indicating a client error. When client errors occur, ASP.NET